3. Funkcje przeciążone, inline i rekurencyjne

Lekcja rozwija temat funkcji o mechanizmy, które pozwalają pisać bardziej elastyczny i czytelny kod.

- funkcje przeciążone - ta sama nazwa, różne parametry
- funkcje inline - małe funkcje zastępowane kodem przez kompilator
- funkcje rekurencyjne - funkcja wywołuje samą siebie
- warunek zakończenia rekurencji

Przeciążanie funkcji

Przeciążenie funkcji polega na zdefiniowaniu kilku funkcji o tej samej nazwie, ale różniących się listą parametrów.

```
double srednia(double a, double b) {
    return (a + b) / 2;
}

double srednia(double a, double b, double c) {
    return (a + b + c) / 3;
}
```

Jak kompilator wybiera funkcję?

- sprawdza nazwę funkcji
- analizuje liczbę argumentów
- porównuje typy argumentów z parametrami
- wybiera najlepiej pasującą definicję

Nie wystarczy zmienić samego typu zwracanego. Funkcje przeciążone muszą różnić się parametrami.

Przykład przeciążenia

```
int maksimum(int a, int b) {
    return (a > b) ? a : b;
}

double maksimum(double a, double b) {
    return (a > b) ? a : b;
}

cout << maksimum(3, 7);       // wersja int
cout << maksimum(2.5, 1.8);   // wersja double
```

Kiedy przeciążenie ma sens?

- gdy operacja ma ten sam sens logiczny, ale działa na różnych typach danych
- gdy chcemy obsłużyć różną liczbę argumentów
- gdy nazwa funkcji opisuje wspólne działanie

Nie należy przeciążać funkcji tylko po to, aby ukryć zupełnie różne działania pod jedną nazwą.

Funkcje inline

Funkcja inline jest sugestią dla kompilatora, aby w miejscu wywołania wstawić treść funkcji. Celem jest ograniczenie narzutu związanego z wywołaniem bardzo krótkich funkcji.

```
inline double kwadrat(double x) {
    return x * x;
}
```

Inline - zalety i ograniczenia

Zaleta	Ograniczenie
może przyspieszyć wywołanie bardzo małej funkcji	kompilator może zignorować sugestię inline
upraszcza zapis krótkich operacji	zbyt częste użycie może zwiększyć rozmiar kodu

W nowoczesnym C++ kompilator sam często decyduje, co opłaca się wstawić w miejscu wywołania.

Rekurencja

Rekurencja zachodzi wtedy, gdy funkcja w trakcie działania wywołuje samą siebie bezpośrednio lub pośrednio.

- przypadek bazowy - kończy rekurencję
- krok rekurencyjny - sprowadza problem do mniejszego przypadku
- wynik powstaje przez powtarzanie tego samego schematu

Przykład: silnia

```
int silnia(int n) {
    if (n == 0) {
        return 1;
    }
    return n * silnia(n - 1);
}
```

Przykład: NWD algorytmem Euklidesa

```
int nwd(int a, int b) {
    if (b == 0) {
        return a;
    }
    return nwd(b, a % b);
}
```

Rekurencja bezpośrednia i pośrednia

Rodzaj	Opis
bezpośrednia	funkcja wywołuje samą siebie
pośrednia	funkcja A wywołuje funkcję B, a B ponownie prowadzi do wywołania A

Najważniejsze wnioski

- przeciążenie zwiększa elastyczność interfejsu funkcji
- inline stosuje się głównie dla prostych, krótkich funkcji
- rekurencja wymaga warunku zakończenia
- każde wywołanie rekurencyjne powinno przybliżać program do przypadku bazowego
- nie każda rekurencja jest najlepszym rozwiązaniem - czasem prościej użyć pętli